

OBJECTIVE

The objective of this laboratory is to:

1. Implement an arbiter with round robin mode of operation that remembers the last grant
2. Design a more robust arbiter the remembers the last two grants and act upon with fairness and priority to serve everyone the same
3. Validate the arbiter.

Round robin arbiter

A round-robin arbiter is a type of arbitration mechanism used in digital systems to manage access to shared resources (e.g., memory, buses, or communication channels) among multiple competing requests. It ensures that all requesters get equal and fair access in a cyclic order, preventing any single requester from monopolizing the resource, preventing starvation.

Design the dut (device under test)

Instructions: You are required to implement an arbiter and its corresponding testbench in Verilog. The arbiter must process three request signals (req[2:0]) and issue three grant signals (gnt[2:0]) accordingly. Your task is to implement the arbiter and create a testbench that verifies the implementation. Follow the details provided below.

Requirements**Arbiter Implementations (Round-Robin Arbiter):**

- The arbiter should prioritize requests in a round-robin manner (equal service to each of the requesters). When two or more requests are asserted simultaneously the arbiter remembers who needs to be served first according to history of who requested and received grant last. In this case the arbiter remembers **only the last grant given**.
 1. The one received grant last will be last in line to get it next – the two others will have priority over him.
 2. The state machine remembers the last grant that was given
 3. Requests are being kept asserted until a grant is issued
- Examples:
 1. If req[0] and req[1] are asserted together, and the last grant was gnt[0], then gnt[1] should be issued next.
 2. If all requests are asserted together, and last one grant was gnt[1], then the grant goes to gnt[0]. For the next cycle, grant will go to gnt[1] with the following rule:

1. Req0 → Req1 → Req2 → Req0 → Req1 → Req2 → ...

- When there are no requests issued, there are no grants (all zero)
- Assume reset (RST signal) is active low and asynchronous
- Use parameters to name the different states you use. Use state RST=000, S_0=001 (for request req[0]) and so on.

Deliverables

1. Verilog code for the **Round-Robin Arbiter**.
2. State machine diagrams for each implementation, showing transitions based on input conditions.
3. Truth table of the state machine as a function of request inputs, current state and the resulted 3 grant outputs. Assume that for each state, all 8 input combinations or request are possible.
 - If you are not using state machine, please clearly explain your design.
4. A Verilog testbench:
 - The testbench should stimulate the arbiter with the following combinations of requests:
 - Req[2:0] = 000
 - Req[2:0] = 001
 - Req[2:0] = 010
 - Req[2:0] = 100
 - Req[2:0] = 111
 - Req[2:0] = 110
 - Req[2:0] = 100
 - Req[2:0] = 000
 - Req[2:0] = 111
 -
 - Clear printouts of grants (gnt[2:0]) at each clock cycle with the corresponding requests and the state names for each

5. Smart(er) arbiter:

Design an arbiter that gets 3 requests from different sources, and output 3 grants one at a time (as in above). If only one is requesting, the grant goes to him in the next cycle and stays asserted until someone else is requesting. If more than one is requesting, the arbiter is a round robin and will fairly serve all request the same priority one at a time. The arbiter will keep the last **two** grants and will consider those in order to decide who gets the next grant keeping fairness for all requesters. A new decision is done every cycle. So if for example the last two grants (two cycles in a row) were given to

requester 0, then requester 1 or 2 will get a grant, and if both (1 and 2) are requesting the lower number gets it first (in this case requester 1 gets a grant and in the next cycle requester 2 will get grant).

Please use two two-bit registers name `last_one_gnt` and `last_two_gnt`. `last_one_gnt` stores the grant that was given last (a number of 0,1,2) , and `last_two_gnt` stores the grant that was given before the last one (number of 0,1,2). At reset time, since we want to prefer lower number requester gets grant if no conflicts and there is still no history, we set the values of `last_one_gnt` to 2 and `last_two_gnt` to 1. Obviously if no grant was given the values of these two registers (`last_one_gnt` and `last_two_gnt`) will be unchanged. Fairness is king so it has first priority over any other rule.

Again, if history will not help us to make clear decision (like the same requester got grant two cycles in a row, so `last_one_gnt` and `last_two_gnt` are equal, the lower number requester will get the grant first if the two others requesting together).

1. **Submit:**

- State transitions diagram for the arbiter. Use One-hot scheme
- Verilog files: `arb.v`, `arb_tb.v`
- `Smart_arb.v` and `smart_arb_tb.v`
- Printout of the results
- Waveforms images.

PART 2 - Datapath, Control, and Finite State Machines

OBJECTIVE

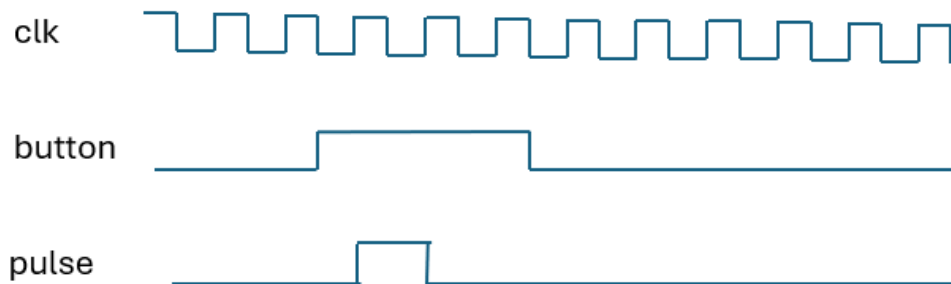
The objective of this laboratory is to:

1. Implement edge detectors (rising or falling edge)
2. Design simple state machines
3. See Ashenden 4.3 pp. 175-181, 184-186

Rising edge detector

Draw a schematic of a circuit that creates a rising edge detector. This is a control signal that emits a 1 clock cycle pulse on the rising edge of a control signal (like a button press). Note that the control signal (push a button, "button" in the drawing below) could be on for much longer than a single clock cycle.

Assume button changes at falling edge of clk. Example:



1. Implement the above circuit (rising edge detector) as a Verilog/SV model (if you need, use `button_dd` and `button_d`, where `d` is delay one cycle, `dd` two cycles)
2. Draw a schematic for a falling edge detector
3. Implement a falling-edge detector in Verilog/SV

Input sequence

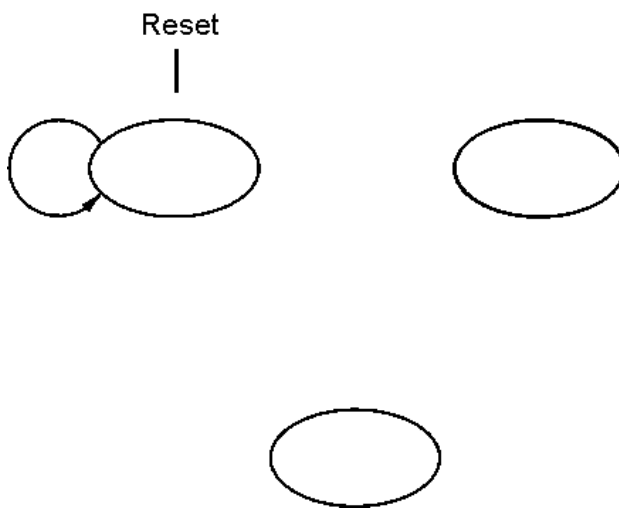
- 1) We want to design a circuit with the following specifications:
 - a) The circuit has one input, `w`, and one output, `z`
 - b) All changes in the circuit occur on the positive edge of a clock signal.

- c) The output z is equal to 1 if, during two immediately preceding clock cycles, the input w was equal to 1. Otherwise, the value of z is 0.

Fill in the output of the circuit given the following input.

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :											

4. Complete the State Diagram (Moore) for the above circuit



- 6) Given the following specification draw a new state diagram.
- The circuit has two inputs, w and x , and one output z .
 - The output z is 1 when the inputs have been opposite for three cycles in a row.
- 7) Create a state diagram for a system that can detect the input sequence '1101'

Deliverables

- Verilog code for 1,3,5,6
- State machine diagrams for each implementation, showing transitions based on input conditions.
- A Verilog testbench for 1,3,5,6:

- State transitions diagram for 1,3,5,6,7
- Verilog files:
- Printout of the results
- Waveforms images.